

Motion Planning Comparison: OMPL, Cartesian and Joint Interpolation

Prepared by Tarunjeet Singh, Project Lead

1. Purpose

This document compares three motion planning approaches implemented within one MoveIt Task Constructor pipeline for the xArm 7 [1], [2]. The aim is to give the team an evidence based way to match each stage of the battery extraction task to the planner that suits it, before moving from simulation to hardware.

2. Test Setup

All three cases used the same package, scene, ten stage task and execution settings. Only the solver passed to the stages changed, selected at run time with the planner parameter.

Execution ran on the simulated xArm 7 through ros2_control fake hardware [3], [4].

Setting	Value
Task	Above battery, descend, attach, lift, transfer, lower, detach, retreat, home
Scene	Battery at (0.40, 0.00) m and drop zone at (0.30, -0.30) m in link_base
Speed	Velocity and acceleration scaling factor 0.15
Platform	Ubuntu 24.04 in WSL2, ROS 2 Jazzy, MoveIt 2, MTC Jazzy
Metrics	Planning time, MTC solution cost, execution time, joint path length

3. Planner Summary

Case	MTC solver	Tool path	Obstacles	Repeatable
1	PipelinePlanner (OMPL)	Free, not constrained	Searches around them	No, stochastic
2	CartesianPath	Straight line	Fails if blocked	Yes
3	JointInterpolationPlanner	Curved, not controlled	Fails if blocked	Yes

4. Results on the Final Task

Values for Case 1 are the mean of two successful runs. Cases 2 and 3 have one successful run each. Case 2 runs that failed because of a configuration error are excluded here and documented in the Case 2 document.

Case	Runs	Plan (ms)	Cost	Exec (s)	Joint path (rad)
1 OMPL	2	594.8	11.717	14.381	5.928
2 Cartesian	1	308.2	11.899	14.647	5.972
3 Joint	1	421.7	11.655	14.251	5.786

Planning time is the wall clock time of `task.plan()`, including scene retrieval. Execution time excludes the demonstration pauses.

5. Run to Run Variation

The initial four stage task, run three times with OMPL at full speed, shows the effect of random sampling on an identical problem.

OMPL run	Plan (ms)	Cost	Exec (s)	Joint path (rad)
1	483.3	21.768	4.561	11.076
2	525.2	4.030	1.785	2.774
3	474.8	4.031	1.785	2.775

Run 1 produced a joint path about four times longer than runs 2 and 3. Cartesian and joint interpolation contain no random sampling, so they return the same path for the same inputs.

6. Discussion

- **Motion quality:** joint interpolation gave the shortest joint path and execution time, OMPL was within 2.5 percent of it, and Cartesian was about 3 percent longer because it holds the tool on straight lines.
- **Planning time:** Cartesian was fastest in this test. With one or two runs per case and scene retrieval included in the measurement, the planning time differences are indicative only.
- **Reliability:** OMPL is the only approach that can find a way around obstacles, but its output varies between runs. Cartesian and joint interpolation are repeatable but fail outright when their direct path is blocked or infeasible.
- **Fit to battery extraction:** a straight vertical approach and retreat matters most near the device, where the tool must enter and leave the battery cavity cleanly. Free space transfers matter less for path shape and more for collision avoidance.

7. Recommendation

A hybrid pipeline assigns each stage to the planner that suits it. MTC supports this directly, because each stage can take its own solver.

Stage	Recommended planner	Reason
Move above battery	OMPL	Free space motion that may need obstacle avoidance
Descend and lift	Cartesian	Straight vertical line into and out of the cavity
Transfer to drop zone	OMPL	Longer free space motion in a cluttered cell
Lower and retreat	Cartesian	Controlled straight line placement
Return home	Joint interpolation	Shortest, repeatable motion through known clear space

8. Limitations and Next Steps

- Increase the sample to at least ten runs per case and report mean and standard deviation.
- Add an obstacle between the battery and the drop zone to test obstacle handling directly.
- Time the solver separately from scene retrieval to isolate true planning cost.
- Implement and evaluate the hybrid pipeline as a fourth case.
- Replace the fixed battery pose with detected poses from the Computer Vision sub-team.
- Validate the selected configuration on the physical xArm 7 in the RIoT Lab.

9. References

- [1] MoveIt Contributors, "Pick and place with MoveIt Task Constructor," MoveIt 2 Documentation. [Online]. Available: https://moveit.picknik.ai/main/doc/tutorials/pick_and_place_with_moveit_task_constructor/pick_and_place_with_moveit_task_constructor.html. Accessed: Sep. 28, 2026.
- [2] M. Görner, R. Haschke, H. Ritter, and J. Zhang, "MoveIt! Task Constructor for task level motion planning," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), Montreal, Canada, 2019, pp. 190–196.
- [3] UFACTORY, "xarm_ros2: ROS 2 developer packages for robotic products from UFACTORY," GitHub, jazzy branch. [Online]. Available: https://github.com/xArm-Developer/xarm_ros2. Accessed: Sep. 28, 2026.
- [4] ros2_control Contributors, "ros2_control Documentation (Jazzy)." [Online]. Available: <https://control.ros.org/jazzy/index.html>. Accessed: Sep. 28, 2026.
- [5] I. A. Şucan, M. Moll, and L. E. Kavraki, "The Open Motion Planning Library," IEEE Robotics & Automation Magazine, vol. 19, no. 4, pp. 72–82, Dec. 2012.
- [6] MoveIt Contributors, "MoveIt 2 Documentation." [Online]. Available: <https://moveit.picknik.ai/main/index.html>. Accessed: Sep. 28, 2026.
- [7] Open Robotics, "moveit_task_constructor API documentation (ROS 2 Jazzy)." [Online]. Available: https://docs.ros.org/en/ros2_packages/jazzy/api/moveit_task_constructor_demo/. Accessed: Sep. 28, 2026.
- [8] Open Robotics, "ROS 2 Documentation: Jazzy Jalisco." [Online]. Available: <https://docs.ros.org/en/jazzy/>. Accessed: Sep. 28, 2026.